SAÉ 1.05 : TRAITER DES DONNÉES

Martin Pépin

novembre/décembre 2025
BUT RT 1A — Ifs — SAÉ1.05

SAE1.05

La SAE1.05 propose de réaliser un petit projet de développement via différentes étapes pour centraliser des données et les mettre en forme.

Elle mobilise les compétences acquises de :

- R106 Architecture des systèmes numériques et informatiques
- R107 Fondamentaux de la programmation
- R108 Bases des systèmes d'exploitation
- R109 Introduction aux technologies Web
- R110 Anglais technique 1
- R111 Expression-Culture-Communication Professionnelles : Introduction à la communication et au savoir-être professionnels
- R112 Projet Personnel et Professionnel
- R115 Gestion de projet 1 : Maîtriser les bases de l'organisation du travail

Rappel des coefficients

Coefficients du Tronc Commun au S1					
		RT1 - Administrer	RT2 - Connecter	RT3 - Programmer	Total toutes compétences
SAÉ1.01	Se sensibiliser à l'hygiène informatique et à la...	10			
SAÉ1.02	S'initier aux réseaux informatiques	31			
SAÉ1.03	Découvrir un dispositif de transmission		36		
SAÉ1.04	Se présenter sur Internet			8	
SAÉ1.05	Traiter des données			35	
SAÉ1.PORTFOLIO	Portfolio	0	0	0	
R1.01	Initiation aux réseaux informatiques	13	4	4	
R1.02	Principes et architecture des réseaux	12			
R1.03	Réseaux locaux et équipements actifs	7	2	2	
R1.04	Fondamentaux des systèmes électroniques	8	8		
R1.05	Supports de transmission pour les réseaux		6		
R1.06	Architecture des systèmes numériques et...	5		5	
R1.07	Fondamentaux de la programmation			19	
R1.08	Bases des systèmes d'exploitation	6		6	
R1.09	Introduction aux technologies Web			4	
R1.10	Anglais technique 1	5	5	5	
R1.11	Expression-Culture-Communication...	4	5	5	
R1.12	Projet Personnel et Professionnel	2	2	2	
R1.13	Mathématiques du signal	5	9		
R1.14	Mathématiques des transmissions	5	9		
R1.15	Gestion de projet 1 : Maîtriser les bases de...		3	3	
	Total sur les SAEs	**41**	**36**	**43**	120
	Total sur les ressources	**72**	**53**	**55**	180
	Total sur les SAEs et les ressources	**113**	**89**	**98**	300

Problématique professionnelle (extrait PN)

Le professionnel R&T est régulièrement amené à **traiter des données** provenant du système d'information de l'entreprise pour ses besoins personnels ou ceux de ses collaborateurs. Ces données peuvent par exemple être liées à l'infrastructure de son réseau (état des équipements, des machines) ou aux utilisateurs.

Généralement **obtenues sous forme brutes**, elles sont ensuite **traitées** avec des objectifs très variés (nettoyage des données, extraction d'informations comptables, archivage, ...) pour être **réutilisées** à d'autres fins ou être **présentées** dans des vues synthétiques.

Ces traitements peuvent être récurrents (mensualisation de bilan, sauvegarde de données périodique, ...) et **gagnent à être automatisés**. Le professionnel R&T doit donc développer des scripts ou des programmes pour gérer de façon efficace le traitement de ces données.

En pratique…

Avec vos compétences, vous devez être capables de :

- créer des « petits » programmes…

En pratique…

Avec vos compétences, vous devez être capables de :

- créer des « petits » programmes…
- pour automatiser le traitement de données…

En pratique…

Avec vos compétences, vous devez être capables de :

- créer des « petits » programmes…
- pour automatiser le traitement de données…
- et produire des statistiques.

En pratique…

Avec vos compétences, vous devez être capables de :

- créer des « petits » programmes…
- pour automatiser le traitement de données…
- et produire des statistiques.

L'objectif est dans notre cas le suivant :

En pratique…

Avec vos compétences, vous devez être capables de :

- créer des « petits » programmes…
- pour automatiser le traitement de données…
- et produire des statistiques.

L'objectif est dans notre cas le suivant :

1. Récolter des données via une API en ligne (OSM)

En pratique…

Avec vos compétences, vous devez être capables de :

- créer des « petits » programmes…
- pour automatiser le traitement de données…
- et produire des statistiques.

L'objectif est dans notre cas le suivant :

1. Récolter des données via une API en ligne (OSM)
2. Mettre en forme les données selon les informations désirées

En pratique…

Avec vos compétences, vous devez être capables de :

- créer des « petits » programmes…
- pour automatiser le traitement de données…
- et produire des statistiques.

L'objectif est dans notre cas le suivant :

1. Récolter des données via une API en ligne (OSM)
2. Mettre en forme les données selon les informations désirées
3. Effectuer des traitements statistiques

En pratique…

Avec vos compétences, vous devez être capables de :

- créer des « petits » programmes…
- pour automatiser le traitement de données…
- et produire des statistiques.

L'objectif est dans notre cas le suivant :

1. Récolter des données via une API en ligne (OSM)
2. Mettre en forme les données selon les informations désirées
3. Effectuer des traitements statistiques
4. Exporter les stats via un format particulier (Markdown)

En pratique…

Avec vos compétences, vous devez être capables de :

- créer des « petits » programmes…
- pour automatiser le traitement de données…
- et produire des statistiques.

L'objectif est dans notre cas le suivant :

1. Récolter des données via une API en ligne (OSM)
2. Mettre en forme les données selon les informations désirées
3. Effectuer des traitements statistiques
4. Exporter les stats via un format particulier (Markdown)

Avec du temps et de l'expérience, on peut aboutir à ce genre de projet (parfait) de collecte et mise en forme de données :
`https://tif.hair/`

Organisation et notation

- 1h de CM

- 2 points d'étape à mi-projet et en fin de projet avec une évaluation de votre avancement
- 13h (ALT) / 20h (INIT) en autonomie pour réaliser le travail demandé avec un cahier des charges donné (projet en binôme)
- Note finale = moyenne entre note du projet et note de suivi

Plan

Introduction

API et formats de données

Le format Markdown

Le projet

Conclusion

Prérequis

Vous avez déjà un bagage technique en Python.

Prérequis

Vous avez déjà un bagage technique en Python.

(Vous connaissez déjà les listes, dictionnaires, etc.)

Prérequis

Vous avez déjà un bagage technique en Python.

(Vous connaissez déjà les listes, dictionnaires, etc.)

Mais avant de partir dans le vif du sujet, nous avons besoin d'introduire quelques pré-requis techniques supplémentaires :

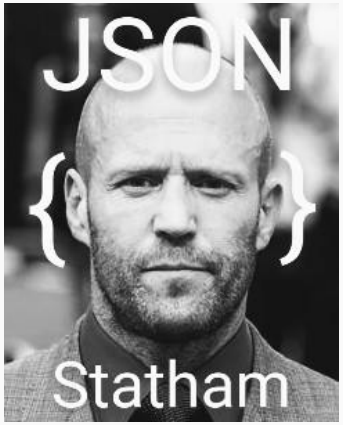
Prérequis

Vous avez déjà un bagage technique en Python.

(Vous connaissez déjà les listes, dictionnaires, etc.)

Mais avant de partir dans le vif du sujet, nous avons besoin d'introduire quelques pré-requis techniques supplémentaires :

- interface de programmation (API)
- modèle de données JSON et XML
- modèle de communication RESTful



API

API (*Application Programming Interface*) : ensemble d'outils qui permet à deux applications de communiquer entre elles.

API

API (*Application Programming Interface*) : ensemble d'outils qui permet à deux applications de communiquer entre elles.

Une API fournit une manière d'accéder aux données d'une application :

- comment interroger l'application ?
- quels sont les paramètres requis et optionnels ?
- quel est le format des données renvoyées par le serveur ?

API

API (*Application Programming Interface*) : ensemble d'outils qui permet à deux applications de communiquer entre elles.

Une API fournit une manière d'accéder aux données d'une application :

- comment interroger l'application ?
- quels sont les paramètres requis et optionnels ?
- quel est le format des données renvoyées par le serveur ?

Exemple d'utilisation d'une API :

`http://api.steampowered.com/ISteamNews/GetNewsForApp/v0002/?appid=730&count=3&maxlength=300&format=json`

API REST

Les **API REST** (*representational state transfer*) sont basées sur le protocole HTTP et ses différentes méthodes afin d'interagir avec un serveur et manipuler ses données :

HTTP Request

Request Headers

```
GET /v1/people/me HTTP/2
Host: webexapis.com
User-Agent: curl/7.64.1
Accept: */*
Authorization:Bearer NDcwYTFhN2MtNjY4My00...-417f-9974-ad72cae0e10f
```

HTTP Response

Response Headers

```
HTTP/2 200
date: Thu, 01 Apr 2021 15:48:15 GMT
content-type: application/json;charset=UTF-8
cache-control: no-cache, no-store
via: 1.1 linkerd
trackingid: ROUTER_6065EB3F-7212-01BB-03CF-AC12F89B03CF
server: Redacted
vary: accept-encoding
strict-transport-security: max-age=63072000; includeSubDomains; preload
```

<blank line>

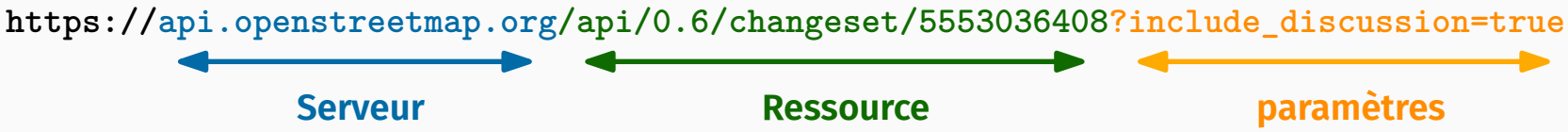
Response Payload


```
{
   "id": "Y2lzY29zcGFyazovL3VzL1...xNzYtYjU1Ny00NjY3NDFlMDlkYjY",
   "emails": [   "cisconian@cisco.com"    ],
   "phoneNumbers": [ {  "type": "mobile",  "value": "+1 817-867-5309"},... ],
   "displayName": "Webex Cisconian",
   "nickName": "Webby",
   "firstName": "Webex",
   "lastName": "Cisconian",
   "avatar": "https://avatar-prod-us-east-2.webexcontent.com/Avtr~V1~1eb65fdf",
   "orgId": "Y2lzY29zcGFyF..LTQxN2YtOTk3NC1hZDcbY2FlMGUxMGY",
   "created": "2016-04-01T15:47:34.726Z",
   "lastModified": "2021-03-24T19:57:59.841Z",
   "lastActivity": "2021-03-26T16:18:49.036Z",
   "status": "active",
   "type": "person",
   "xmppFederationJid": "cisconian@cisco.com"
}
```


This data is all exchanged as simple text over a TCP/TLS connection.

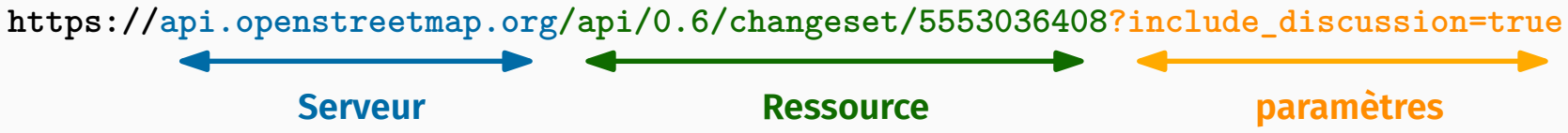
API REST

Une requête REST a plusieurs paramètres, certains optionnels :



API REST

Une requête REST a plusieurs paramètres, certains optionnels :



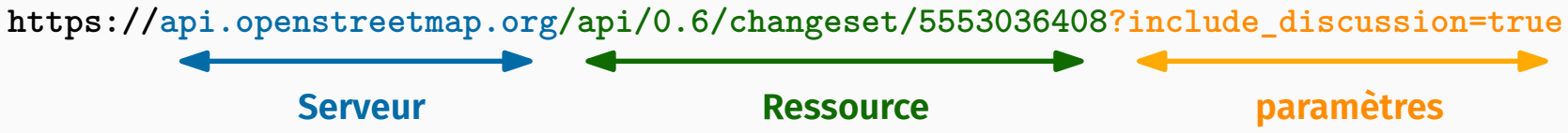
On peut utiliser plusieurs méthodes, et les requêtes produisent différents codes de retour

HTTP Verb	Typical Action (CRUD)
`POST`	Create
`GET`	Read
`PUT`	Update
`PATCH`	Update
`DELETE`	Delete

Status Code	Status Message	Meaning
200	OK	All looks good
201	Created	New resource created
400	Bad Request	Request was invalid
401	Unauthorized	Authentication missing or incorrect
403	Forbidden	Request was understood but not allowed
404	Not Found	Resource not found
500	Internal Server Error	Something wrong with the server
503	Service Unavailable	Server is unable to complete request

API REST

Une requête REST a plusieurs paramètres, certains optionnels :



On peut utiliser plusieurs méthodes, et les requêtes produisent différents codes de retour

HTTP Verb	Typical Action (CRUD)
`POST`	Create
`GET`	Read
`PUT`	Update
`PATCH`	Update
`DELETE`	Delete

Status Code	Status Message	Meaning
200	OK	All looks good
201	Created	New resource created
400	Bad Request	Request was invalid
401	Unauthorized	Authentication missing or incorrect
403	Forbidden	Request was understood but not allowed
404	Not Found	Resource not found
500	Internal Server Error	Something wrong with the server
503	Service Unavailable	Server is unable to complete request

Dans certains cas, les API REST demandent une clé d'authentification

Format de données

En utilisant une API, un programme peut interagir avec un autre programme. Cependant, le format des données renvoyé n'est pas toujours standard !

Format de données

En utilisant une API, un programme peut interagir avec un autre programme. Cependant, le format des données renvoyé n'est pas toujours standard !

Prenons par exemple le résultat d'une commande en CLI Cisco pour afficher la liste des interfaces réseau :

```
switch# show the int bri

--------------------------------------------------------------------------------
Ethernet      VLAN    Type Mode   Status  Reason                   Speed     Port
Interface                                                                     Ch #
--------------------------------------------------------------------------------
Eth1/1        1       eth  fabric down    SFP not inserted            10G(D) --
```

Format de données

En utilisant une API, un programme peut interagir avec un autre programme. Cependant, le format des données renvoyé n'est pas toujours standard !

Prenons par exemple le résultat d'une commande en CLI Cisco pour afficher la liste des interfaces réseau :

```
switch# show the int bri

--------------------------------------------------------------------------------
Ethernet      VLAN    Type Mode   Status  Reason                   Speed     Port
Interface                                                                     Ch #
--------------------------------------------------------------------------------
Eth1/1        1       eth  fabric down    SFP not inserted            10G(D) --
```

Le format renvoyé par l'IOS Cisco est en texte clair : conçu pour être lu par un humain. Ce format n'est pas du tout adapté aux outils de programmation ! Où sont les mots-clés ? Où est l'information utile ?

Format de données : XML

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<nf:rpc-reply xmlns:nf="urn:ietf:params:xml:ns:netconf:base:1.0" xmlns="http://www.cisco.com/nxos:1.0:if_manager">
 <nf:data>
  <show>
   <interface>
    <__XML__INTF_ifeth_brf>
     <__XML__PARAM_value>
      <__XML__INTF_output>Ethernet1/1-3</__XML__INTF_output>
     </__XML__PARAM_value>
     <__XML__OPT_Cmd_show_interface_if_eth_brief___readonly__>
      <__readonly__>
       <TABLE_interface>
        <ROW_interface>
         <interface>Ethernet1/1</interface>
         <vlan>1</vlan>
         <type>eth</type>
         <portmode>fabric</portmode>
         <state>down</state>
         <state_rsn_desc>SFP not inserted</state_rsn_desc>
         <speed>10G</speed>
         <ratemode>D</ratemode>
        </ROW_interface>
       </TABLE_interface>
      </__readonly__>
     </__XML__OPT_Cmd_show_interface_if_eth_brief___readonly__>
    </__XML__INTF_ifeth_brf>
   </interface>
  </show>
 </nf:data>
</nf:rpc-reply>
]]>]]>
```

XML (*eXtensible Markup Language*) est un format de données organisé sous forme de balises. Il est fortement structuré, ce qui évite les malformations et qui permet de vérifier l'intégrité des données.

Format de données : XML

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<nf:rpc-reply xmlns:nf="urn:ietf:params:xml:ns:netconf:base:1.0" xmlns="http://www.cisco.com/nxos:1.0:if_manager">
 <nf:data>
  <show>
   <interface>
    <__XML__INTF_ifeth_brf>
     <__XML__PARAM_value>
      <__XML__INTF_output>Ethernet1/1-3</__XML__INTF_output>
     </__XML__PARAM_value>
     <__XML__OPT_Cmd_show_interface_if_eth_brief___readonly__>
      <__readonly__>
       <TABLE_interface>
        <ROW_interface>
         <interface>Ethernet1/1</interface>
         <vlan>1</vlan>
         <type>eth</type>
         <portmode>fabric</portmode>
         <state>down</state>
         <state_rsn_desc>SFP not inserted</state_rsn_desc>
         <speed>10G</speed>
         <ratemode>D</ratemode>
        </ROW_interface>
       </TABLE_interface>
      </__readonly__>
     </__XML__OPT_Cmd_show_interface_if_eth_brief___readonly__>
    </__XML__INTF_ifeth_brf>
   </interface>
  </show>
 </nf:data>
</nf:rpc-reply>
]]>]]>
```

XML (*eXtensible Markup Language*) est un format de données organisé sous forme de balises. Il est fortement structuré, ce qui évite les malformations et qui permet de vérifier l'intégrité des données. HTML est dérivé de XML, c'est assez similaire !

Format de données : JSON

```
{
    "interfaces": [
        {
            "interface": "Ethernet1/1",
            "vlan": "1",
            "type": "eth",
            "portmode": "fabric",
            "state": "down",
            "state_rsn_desc": "SFP not inserted",
            "speed": "10G",
            "ratemode": "D"
        }
    ]
}
```

JSON (*JavaScript Object Notation*) est un format de données alternatif où les données sont moins structurées, mais où la lisibilité humaine est beaucoup plus accessible.

Exploitation d'une API

Dans le cadre de cette SAE, nous travaillerons avec deux API REST.

Il existe plusieurs façon d'interagir avec :

Exploitation d'une API

Dans le cadre de cette SAE, nous travaillerons avec deux API REST.

Il existe plusieurs façon d'interagir avec :

- Requêtes HTTP directement via un navigateur :
  - ✓ pratique pour tester / expérimenter,
  - ✗ on ne récupère pas les données pour les traiter plus tard ;

Exploitation d'une API

Dans le cadre de cette SAE, nous travaillerons avec deux API REST.

Il existe plusieurs façon d'interagir avec :

- Requêtes HTTP directement via un navigateur :
  - ✓ pratique pour tester / expérimenter,
  - ✗ on ne récupère pas les données pour les traiter plus tard ;

- Frontend web *user-fiendly* :
  - ✓ pratique pour tester / expérimenter,
  - ✗ impossible d'automatiser ;

Exploitation d'une API

Dans le cadre de cette SAE, nous travaillerons avec deux API REST.

Il existe plusieurs façon d'interagir avec :

- Requêtes HTTP directement via un navigateur :
  - ✓ pratique pour tester / expérimenter,
  - ✗ on ne récupère pas les données pour les traiter plus tard ;

- Frontend web *user-fiendly* :
  - ✓ pratique pour tester / expérimenter,
  - ✗ impossible d'automatiser ;

- Bibliothèque **`requests`** en Python pour faire des appels HTTP :
  - ✓ on peut récupérer les données,
  - ✓ conversion automatique en objets Pythons,
  - ✓ on peut automatiser.

Bibliothèque `requests`

Exemple d'utilisation :

```python
import requests


api_url = "https://jsonplaceholder.typicode.com/todos/1"
response = requests.get(api_url)
donnees = response.json()
```

Plan

Introduction

API et formats de données

Le format Markdown

Le projet

Conclusion

Markdown : généralités

Markdown est un langage de balisage léger qui permet de mettre en forme des documents texte avec une syntaxe facile à lire et à écrire. Un document au format `.md` est très humain-compatible.

Markdown : généralités

Markdown est un langage de balisage léger qui permet de mettre en forme des documents texte avec une syntaxe facile à lire et à écrire. Un document au format `.md` est très humain-compatible.

On peut « compiler » un fichier Markdown en PDF ou en page HTML.

Markdown : généralités

Markdown est un langage de balisage léger qui permet de mettre en forme des documents texte avec une syntaxe facile à lire et à écrire. Un document au format `.md` est très humain-compatible.

On peut « compiler » un fichier Markdown en PDF ou en page HTML.

Créé en 2004, il est notamment beaucoup utilisé pour les blogs, forums simples, et surtout les outils de développement collaboratifs et autres fichiers README.

par exemple sur GitHub : `https://github.com/sherlock-project/sherlock/blob/master/README.md`

Markdown : généralités

- Prenons par exemple l'écriture de cette phrase en **HTML** :

```
<p> Téma la taille du <strong> rat </strong> </p>
```

Markdown : généralités

- Prenons par exemple l'écriture de cette phrase en **HTML** :

```
<p> Téma la taille du <strong> rat </strong> </p>
```

- On peut l'écrire de la même manière en LaTeX :

```
Téma la taille du  \textbf{rat}
```

Markdown : généralités

- Prenons par exemple l'écriture de cette phrase en **HTML** :

```
<p> Téma la taille du <strong> rat </strong> </p>
```

- On peut l'écrire de la même manière en LaTeX :

```
Téma la taille du  \textbf{rat}
```

Dans les deux cas précédents, les documents vont bien s'afficher comme voulu mais relecture peut être pénible.

Markdown : généralités

- Prenons par exemple l'écriture de cette phrase en **HTML** :

```
<p> Téma la taille du <strong> rat </strong> </p>
```

- On peut l'écrire de la même manière en LaTeX :

```
Téma la taille du  \textbf{rat}
```

  Dans les deux cas précédents, les documents vont bien s'afficher comme voulu mais relecture peut être pénible.

- En **Markdown**, c'est bien plus simple :

```
Téma la taille du **rat**
```

Markdown : syntaxe

Vous irez vous-même découvrir la syntaxe de Markdown :

`https://markdown-guide.readthedocs.io/en/latest/basics.html`

Un petit aperçu quand même :

Markdown : syntaxe

Vous irez vous-même découvrir la syntaxe de Markdown :

`https://markdown-guide.readthedocs.io/en/latest/basics.html`

Un petit aperçu quand même :

- Deux sauts de lignes pour séparer les paragraphes

Markdown : syntaxe

Vous irez vous-même découvrir la syntaxe de Markdown :

https://markdown-guide.readthedocs.io/en/latest/basics.html

Un petit aperçu quand même :

- Deux sauts de lignes pour séparer les paragraphes
- Le gras et l'italique se gèrent avec des étoiles :

```
*Texte en italique*
**Texte en gras**
*** Gras et italique***
```

Markdown : syntaxe

Vous irez vous-même découvrir la syntaxe de Markdown :

https://markdown-guide.readthedocs.io/en/latest/basics.html

Un petit aperçu quand même :

- Deux sauts de lignes pour séparer les paragraphes
- Le gras et l'italique se gèrent avec des étoiles :

```
*Texte en italique*
**Texte en gras**
*** Gras et italique***
```

- Les titres sont définis avec des dièses :

```
# titre principal (h1)
## sous titre (h2)
### sous-sous titre (h3)
#### ...
```

Markdown : syntaxe

- Listes ordonnées et non ordonnées (pour les sous-listes, indentez de 4 espaces) :

```
1. Premier élément
2. Deuxième élément
    - sous-élément
    - autre sous-élément
```

- Pour insérer une image :

```
![alt text](https://i.imgur.com/Ssns07U.jpeg)
![](./images/bruh.png)
```

- Et plein d'autres outils assez faciles à aborder... Pour se simplifier la vie, on peut tester avec un outil en ligne : `https://markdown-editor.github.io/`

Markdown : en Python

Si Markdown est un langage simple, il faut tout de même utiliser un compilateur pour le transformer dans un autre format. En Python, on peut utiliser la bibliothèque du même nom :

```
import markdown

with open('file.md', 'r') as f:
    text = f.read()

html = markdown.markdown(text)

with open('file.html', 'w') as f:
    f.write(html)
```

Plan

Introduction

API et formats de données

Le format Markdown

Le projet

Conclusion

Plan

Introduction

API et formats de données

Le format Markdown

Le projet

Conclusion

Conclusion

Quelques nouvelles notions :

- Les API, et plus particulièrement les API REST
- Les formats de donnée XML et JSON
- Comment interagir avec une API
- Le langage Markdown

Conclusion

Quelques nouvelles notions :

- Les API, et plus particulièrement les API REST
- Les formats de donnée XML et JSON
- Comment interagir avec une API
- Le langage Markdown

Donc en bref,

- pas beaucoup de nouveautés, du moins rien de très compliqué

Conclusion

Quelques nouvelles notions :

- Les API, et plus particulièrement les API REST
- Les formats de donnée XML et JSON
- Comment interagir avec une API
- Le langage Markdown

Donc en bref,

- pas beaucoup de nouveautés, du moins rien de très compliqué
- le but de cette SAÉ est d'aller plus loin que dans les ressources classiques et de réaliser un vrai projet de programmation.

Conclusion

Quelques nouvelles notions :

- Les API, et plus particulièrement les API REST
- Les formats de donnée XML et JSON
- Comment interagir avec une API
- Le langage Markdown

Donc en bref,

- pas beaucoup de nouveautés, du moins rien de très compliqué
- le but de cette SAÉ est d'aller plus loin que dans les ressources classiques et de réaliser un vrai projet de programmation.

À vous de jouer !